2025 IEEE International Conference on Contemporary Computing and Communications (InC4)

From AKKA CQRS to Kafka Streams: A Case Study in Real-Time Data Transformation

Hatim Palitanawala
Mumbai University
Phoenix, Arizona
hellohatim@gmail.com

Ramprasad Goturkanam
VTU University
Boston, MA
ramprasadgk75@gmail.com

Ali Akbar Hathiyari
JP Morgan Chase
Mumbai, India
ahathiyari@gmail.com

Shabbir Kapasi
Mumbai University
Mumbai, India
shabbir.fkapasi@gmail.com

Hussain Suttarwala
Mumbai University
Phoenix, Arizona
hm_suttarwala@hotmail.com

2025 IEEE International Conference on Contemporary Computing and Communications (InC4) | 979-8-3315-2118-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/InC465408.2025.11256412

Abstract - This paper delves into migrating from an Akka-based CQRS (Command Query Responsibility Segregation) architecture to Kafka Streams for real-time data processing in a distributed system. In a traditional implementation, the Akka CQRS model is used for complex state management and event-sourced systems, providing flexibility and scalability for handling read-write separation in microservices. However, the need for real-time, high-throughput, and low-latency processing eventually led to the shift to Kafka Streams. We explore the strengths and limitations of Akka in CQRS implementations and analyze how Kafka Streams can simplify and optimize event-driven architectures. In this paper, we present insights into the impact of this architectural shift by examining key use cases, such as handling data from multiple topics, transforming data, persisting to databases, and ensuring reliable message delivery. We also discuss various windowing techniques, including punctuating with state stores and handling edge cases like publishing zero-count messages when data is absent from the stream. We will visit the performance comparisons of the two frameworks, wherein we have observed zero latency from Kafka and almost ~3% improvement in throughput with improved resource utilization compared to AKKA.

Keywords— Akka, Actor model, Kafka streaming, sensor data, Windowing techniques

I. Introduction

In modern software systems, achieving scalability and responsiveness often requires a clear separation of concerns between write operations (commands) and read operations (queries). The Command Query Responsibility Segregation (CQRS) pattern is a popular architectural approach to address this by isolating the write and read sides of the application, allowing each to be optimized independently. Traditionally, relational databases like Postgres are used on the write side due to their strong transactional guarantees and ability to ensure data consistency. On the read side, NoSQL databases like MongoDB are often used to optimize fast reads, enabling queries on denormalized datasets and providing scalability through horizontal sharding.

In a typical CQRS setup, Postgres handles all write operations, ensuring strict ACID compliance and managing complex relationships between entities. The read side of the architecture, typically backed by MongoDB, provides fast, efficient access to data that has been pre-aggregated or denormalized to match the application's query patterns. This architecture is beneficial in systems where the volume of reads significantly exceeds writes or is different. Optimization strategies are required for reading and writing data.

However, as systems grow and demand for real-time data processing increases, the traditional CQRS pattern can face challenges, especially in high-throughput environments requiring real-time event processing and immediate data availability across multiple services. Kafka Streams, a distributed stream-processing framework, offers an alternative approach to address these challenges by enabling real-time transformation and processing of data as it flows through Kafka topics. Leveraging Kafka Streams allows data to be processed, transformed, and written to a database or consumed by other services in real time without needing separate read/write models or dedicated services.

This paper presents a case study of a system initially built using CQRS with Postgres and MongoDB, where data was ingested from multiple sources, transformed, and merged into a database for subsequent query by the UI. As the system requirements evolved towards real-time data processing and transformation, Kafka Streams was introduced to replace the CQRS architecture. We explore the reasons for this shift, discuss the architectural changes, and compare the two approaches. Additionally, we highlight the performance gains, the complexity introduced by Kafka Streams, and the situations in which Kafka Streams is more suitable compared to a traditional CQRS pattern. Through this analysis, we aim to offer practical guidance for engineers considering migrating from a CQRS-based architecture to a Kafka Streams-based approach in modern, real-time systems.

II. Background of the Use Case

This section provides background on the business use case where architectural patterns were applied. The use case involves industry machines sending vast amounts of data to an analytics system that will use the information to analyze the data in real-time and provide the feeds to the end users, who can then use that data to override the machine/sensor operations manually. Below are some of the significant nonfunctional requirements in the use case context.

- Minimum latency on write operations.
- High throughput on write operations, including the entire data processing pipeline.
- High volume of data that needs to be handled.
- Nearly real-time response is required from when the machine sensors generate the data to when it is consumed.

979-8-3315-2118-9/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

In the above business use case, we will discuss how AKKA and Kafka Streams were applied and their advantages and disadvantages.

III. Related Work

The section below studies various papers written on Akka and Kafka streams. While some papers provide invaluable insights into the processes and tools used for ingesting large volumes of streaming data, others demonstrate the application of those tools in various use cases related to the current paper's use.

- Blesser et al. [6] describe techniques for testing applications written in the Akka framework. The methods used here can also conceptually be used to test the Kafka streams and complement the knowledge in the paper, especially some aspects of resiliency testing.
- Transparent Actor Model [5] by Ghassemi et al. proposed a pseudo-actor language that can be used as a pseudo-design language to describe actor-based implementations across all implementations, including Akka, ABS, and others. In the context of the current paper, readers can potentially leverage the information to document their design decisions while implementing Akka or other competing frameworks.
- Huber et al. [4] discussed AKKA to mine Reddit Data. This paper exemplifies how AKKA is used in significant data processing use cases. This paper suggests the hierarchical approach of spawning child actors that naturally fits into the AKKA actor model. Such use cases will warrant a need for AKKA compared to Kafka, where the use case naturally fits into the AKKA Architecture
- Sarma et al. [3] discussed agent-based implementation of energy management systems. It is another use case example of how AKKA has been used in industries that consume large amounts of sensor and machine data. Similar to paper [4], this use case also naturally lends itself to the AKKA architecture
- Research publications in simulations By Bytschkow et al. [1] employ AKKA to implement an actor-oriented simulation approach, where ICT components are represented as independent actors. Each actor can send and receive messages, process data, and make autonomous decisions based on the system's state. Apart from the Hierarchical structure that AKKA lends to, additional use case factors include Actor-based simulation and independent Agent-based decision-making process.
- Haidong et al. [2] emphasized the lightweight nature of AKKA actors, stating that a node with 1GB RAM can support approximately 2.5 million actors. This is contrasted with traditional multithreading models, which have limitations regarding the number of threads they can handle concurrently. The paper also exemplifies how AKKA simplifies network communications.
- The work done in [7] provides a detailed analysis of Kafka configurations, which can further complement the paper and provide readers with additional insights to optimize their Kafka setup
- The work done in [8] provides users with alternatives to Kafka that can be used when Kafka is not an option. Figure 9 in this paper [8] compares the alternatives in this paper. This can also help readers consider possible options for Kafka that can provide comparable low latency and high throughput.

Overall, the current research papers show a clear set of use cases, like simulation, etc., where Actor/agent-based modeling is the core need, and AKKA makes for a precise fit. Little is written on Kafka streams, but there are many concepts we can borrow from the existing papers that can be extended to streaming applications.

IV. Akka Implementation

Here, we will describe the original architectural setup, where commands (writes) went to Postgres and queries (reads) went to MongoDB. We will also explain how the system handled real-time data from two Kafka topics, transforming and saving the data to Postgres and MongoDB.

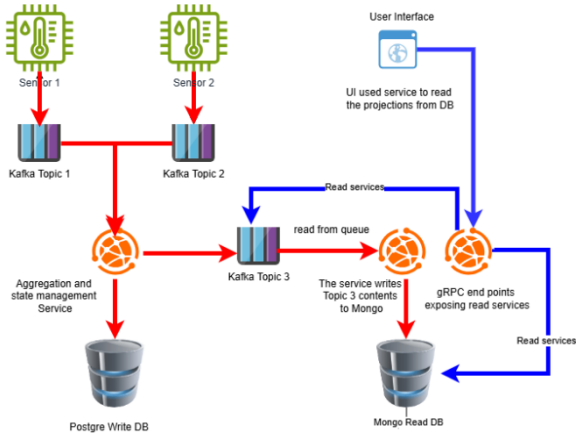


Fig. 1. Akka-based solution

The original architecture relied on Akka to manage the CQRS pattern, where:

- Commands – represented by red lines in Fig. 1, were processed by Akka actors, updating the system's state.
- Queries, represented by blue lines in Fig. 1, were handled through projections stored in MongoDB or PostgreSQL, the final result of the data processing pipeline, which got its feeds from event sourcing used to store the history of state transitions.
- Event sourcing ensured that state changes were logged as immutable events, which could be replayed to restore state or derive projections.

In this setup, two Kafka topics provided real-time data about machines, which were processed using Akka actors and then stored in databases. The UI retrieved data through projections using REST endpoints.

Akka's actor model makes it a compelling choice for implementing CQRS, as it offers:

- Concurrency and scalability: Akka ensures safe and concurrent processing of commands and queries by isolating mutable states in actors. It helps free up the developer's bandwidth from manually handling threads and concurrency issues in the code, thereby

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

letting them concentrate on the core business functionality.

- Event sourcing: Akka easily supports event sourcing, enabling systems to track the history of state changes through immutable events.
- Fault tolerance: The supervision strategy in Akka ensures system resilience by restarting failed actors.

However, as system complexity grows, especially when real-time processing is critical, AKKA's actor model can introduce overhead, subsequently introducing latency, particularly in scenarios with high-throughput data streams. Managing distributed state and windowing mechanisms in real-time becomes challenging when low-latency response times are required.

V. Performance Statistics of the Akka Implementation

The above architecture diagram in Fig. 1 shows how the data processing pipeline aggregates and transforms data from its origination at the source to its consumption in the web application. The data processing pipeline has the following hops.

- Event sourcing via Kafka topics 1 and 2
- Aggregation via the AKKA-based containerized service.
- Storing the aggregation results in Postgres-DB and the Kafka topic 3
- A second service that is a sink for sending the aggregated data to MongoDB and the web application.

The table below illustrates the latency time for a sample of messages from various machines. Topic-1 and Topic-2 store the original event-originating timestamp, while Topic-3 stores the resulting aggregating Topic-3 timestamp. The latency is calculated as

$$Latency = Topic\text{-}3 - Max\ (Topic\text{-}1,\ Topic\text{-}2) \qquad (1)$$

TABLE I. SAMPLESAMPLE MESSAGES IN THE AKKA SYSTEM

Item	Topic-1	Topic-2	Topic-3	Latency (ms)	DB
1	12/20/2024, 11:47:02:53	12/20/2024, 11:47:02:51	12/20/2024, 11:47:03:02	9ms	12/20/2024, 11:47:03:02
2	12/20/2024, 11:47:02:53	12/20/2024, 11:47:02:51	12/20/2024, 11:47:03:02	9ms	12/20/2024, 11:47:03:02
3	12/20/2024, 11:47:02:53	12/20/2024, 11:47:02:51	12/20/2024, 11:47:03:02	9ms	12/20/2024, 11:47:03:02
4	12/20/2024, 11:47:02:53	12/20/2024, 11:47:02:51	12/20/2024, 11:47:03:02	9ms	12/20/2024, 11:47:03:02
1	12/20/2024, 11:47:03:20	NA	12/20/2024, 11:47:03:24	4ms	12/20/2024, 11:47:03:24
2	12/20/2024, 11:4:03:10	12/20/2024, 11:47:03:14	12/20/2024, 11:47:03:14	0	12/20/2024, 11:47:03:14
1	12/20/2024, 11:47:03:50	NA	12/20/2024, 11:47:03:50	0	12/20/2024, 11:47:03:50
3	12/20/2024, 11:47:03:56	NA	12/20/2024, 11:47:03:58	2	12/20/2024, 11:47:03:58
4	12/20/2024, 11:47:03:02	NA	12/20/2024, 11:47:03:02	0	12/20/2024, 11:47:03:02
1	12/20/2024, 11:47:04:10	NA	12/20/2024, 11:47:04:10	0	12/20/2024, 11:47:04:10
1	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
2	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
3	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
4	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
5	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
6	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
7	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
8	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
9	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28
9	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:28	6ms	12/23/2024, 11:48:06:28

From the above statistics, we can infer that the highest latency is observed when both sourcing topics have data aggregated in the AKKA web service application and inserted into Topic-3. So, for records 1 through 4, we see the highest latency. In other messages, where no aggregation is involved, the latency is either less than or zero, but we still see some latency.

Referring to Fig. 1, we see the issues that arise in the areas of architecture below.

- The aggregation and the state management service add another layer of processing, which increases the latency before the UI retrieves the data.
- The layer mentioned above suffers from the limitations of the AKKA-based framework. As the data size grows, the limitations become far more evident.
- We start seeing DB constraints and an increase in throughput.

Overall, the added architectural layers introduced by implementing CQRS via service and DB layers do not scale out as effectively for real-world use cases where high-volume data feeds into near-real-time use case consumers.

VI. Kafka Implementation

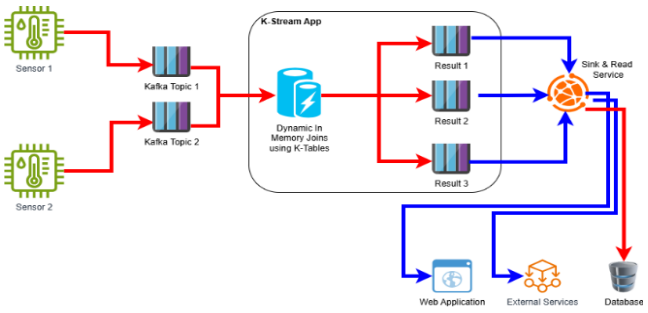


Fig. 2. Kafka Implementation

From our understanding of the key pain areas of the original system, Kafka Streams came out as a suitable choice to replace some of the bottlenecks. Kafka Streams is optimized to handle real-time streaming data directly from Kafka topics. This implementation provides a clean abstraction for:

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

- Built-in fault tolerance and scalability: Leveraging Kafka's partitioning and replication ensures that processing is distributed and resilient.
- Windowing and stateful processing: Kafka Streams natively supports windowed operations, making it well-suited for use cases that require sliding, tumbling, or hopping windows.
- Event-driven architecture: Kafka Streams automatically processes each incoming event, enabling real-time transformations and aggregations without explicit actor-based concurrency management.

The transition to Kafka Streams simplified the pipeline. Key changes included:

- Direct processing of Kafka topics: Kafka Streams consumed real-time data from the two issues, processing and transforming it before storing it in databases (PostgreSQL/MongoDB).
- Stateful processing and windowing: Kafka Streams managed windowed transformations and persisted state using state stores, ensuring efficient aggregation and low-latency updates.
- Automatic scaling: Kafka Streams automatically manages partitioned data, scaling processing across multiple nodes without manual actor-based concurrency.

VII. Kafka windowing techniques

As shown in Fig. 2, the simplified architecture replaces the Akka service performing the aggregations. The most crucial gain here is that the in-memory processing of Kafka streams significantly reduces the latency and improves performance.

Apart from the key performance gains, Kafka Streams provides powerful windowing concepts that allow for processing data in real-time over specified periods, including:

- Tumbling windows: Fixed-sized, non-overlapping windows suitable for aggregating events in discrete periods.
- Sliding windows: These overlapping windows allow for more fine-grained processing and provide updated results as events occur within the window.
- Hopping windows are like sliding windows but have fixed intervals, making them ideal for periodic aggregations.

In our case, we initially attempted to use the suppress operator for windowing, which ensures that results are published once the window closes. However, due to the suppressor operator's event-time-based nature, the system failed to publish data in scenarios where no new events arrived, leading to unpredictable delays in message delivery.

We switched to using punctuation combined with state stores to handle Edge Cases, Punctuation, and State Stores. Punctuation allowed us to trigger periodic computations, ensuring that data was published at the required interval, even if no new data arrived in the stream. This was particularly useful for generating zero-count messages when no events were recorded, providing more predictable output for the UI.

VIII. Kafka implementation performance

This section presents performance statistics after substituting the Akka layer with the Kafka Streams app.

The latency has reduced to zero, as Kafka can instantaneously apply aggregation logic using its in-memory processing.

We also observed that due to reduced latency, the system's overall throughput increased, and the system was able to handle more incoming messages from the increased number of machines.

TABLE II. SAMPLE MESSAGES FROM KAFKA STREAMS

Item	Topic-1	Topic-2	Topic-3	Latency (ms)	DB
1	12/20/2024, 11:47:02:53	12/20/2024, 11:47:02:51	12/20/2024, 11:47:02:53	0	12/20/2024, 11:47:02:53
1	12/20/2024, 11:47:02:53	12/20/2024, 11:47:02:51	12/20/2024, 11:47:02:53	0	12/20/2024, 11:47:02:53
1	12/20/2024, 11:47:02:53	12/20/2024, 11:47:02:51	12/20/2024, 11:47:02:53	0	12/20/2024, 11:47:02:53
1	12/20/2024, 11:47:02:53	12/20/2024, 11:47:02:51	12/20/2024, 11:47:02:53	0	12/20/2024, 11:47:02:53
2	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22
3	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22
4	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22
5	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22
6	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22
7	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22
8	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22
9	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22
9	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	12/23/2024, 11:48:06:22	0	12/23/2024, 11:48:06:22

IX. Comparative analysis

Summarizing some of our findings from above, we conclude that AKKA has advantages:

- Complex state management: Akka excels in handling complex stateful workflows and event sourcing, making it ideal for applications with intricate event-driven logic.
- Concurrency control: Akka's actor model is well-suited for high-concurrency environments with many simultaneous events.

On the other hand, Kafka Streams can be used where :

- Simplicity and scalability: Kafka Streams abstracts much of the complexity of processing real-time streams, automatically handling state, fault tolerance, and windowing.
- Built-in stream processing: Kafka Streams is purpose-built for streaming scenarios, making it more efficient for real-time aggregations and transformations without external actors or projections.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

- Applicability to real-time use cases like publishing zero-count messages. Using “Kafka punctuate” and “state stores,” we ensured that the system would publish zero-count messages during periods of inactivity, providing consistent feedback to the UI.

Apart from the relevance to the business use case, the new simplified architecture also provides additional benefits:

- We see a smaller number of components.
- As such, we also have fewer deployments.
- We see reduced time and cost to develop the system.
- Reduced errors.
- Faster time to market.

While we spoke about the improvements, here are some key points to note:

- The infrastructure should allow Kafka clusters to scale horizontally.
- Since the new architecture shifts to in-memory processing, the new infrastructure should account for the memory-intensive infrastructure requirements accordingly.
- Any security considerations that were earlier implemented via DB access control procedures now need to be implemented within Kafka.
- Appropriate administrative controls must be kept in place to handle Kafka queues.

X. Throughput & Performance comparison

Here, we will discuss the throughput and overall performance comparisons and briefly discuss the infrastructure used to arrive at the throughput.

A. *Test Configuration*

- CPU: 250m
- Memory: 500MB
- Limits: 500m CPU, 1 GB Memory
- Requests: 100m CPU, 128MB Memory

B. *Throughput Comparison. We observe ~3% improvement in Kafka throughput*

TABLE III. Throughput comparison

Time	AKKA (# events)	Kafka (# events)
10 minutes	19,198	23,437
20 minutes	36,909	41,119
60 minutes	104,128	108,045

C. *Consumer Lag:*

- While Table. I and Table. II demonstrate the latency on the write operations, the following shows the effect of the latency on the consumer side lag
- Akka: 300 messages behind consistently
- Kafka: 0 lag consistently

D. *Resource Utilization:*

- Memory: Kafka has around 10 to 15% lower utilization as it effectively uses the RocksDB and Kafka’s internal mechanisms
- AKKA-based CQRS requires higher CPU usage due to additional event store management in MongoDB

E. *Scalability:*

- Kafka-based streams quickly scale by adding more partitions or instances, while AKKA-based CQRS requires more manual orchestration for handling MongoDB and projections

F. *Error handling and fault tolerance:*

- Kafka Streams offers built-in exactly-once semantics and fault-tolerant state management, whereas Akka requires more configuration to ensure reliable event handling and consistency.

XI. Conclusion

The transition from Akka-based CQRS to Kafka Streams for real-time data processing has demonstrated clear advantages in terms of scalability, latency, and simplification of stream processing workflows.

Kafka Streams offer higher throughput and lower latency under load while being more resource-efficient. The gains in latency are substantial, reaching 0, while on throughput, we have consistently observed a minimum 3% improvement. For many use cases, such as monitoring industrial sensors, latency can be a significant concern.

Furthermore, Kafka Streams' ability to scale horizontally with partitioning and tight integration makes them a superior choice for high-throughput, low-latency use cases.

By leveraging Kafka Streams’ native windowing, stateful processing, and integration with Kafka topics, we improved system performance and ensured reliable message delivery even under edge-case scenarios.

While Akka remains a powerful tool for complex, stateful systems, Kafka Streams offers a more streamlined and efficient approach for high-throughput, low-latency processing applications.

While we prescribe Kafka streams in use cases where High throughput and low latency are critical business needs, readers are encouraged to notice the key architectural trade-offs and how they will affect the needs of their specific business use case and project constraints.

While this paper focused on a particular setup and use case, its outcomes can generally be applied to many industrial applications and robotic automation where synthesizing and aggregating data with low latency and high throughput goals is needed.

This paper can be extended by further exploring comparative use cases across other industry sectors. There are more technologies like Mongo time series database offerings and others that can further be explored in the context of high throughput low latency use cases, including multiple other nonfunctional requirements like ease of deployments, maturity of the frameworks, and much more, each of which

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.

can be critical to the application of these software components in large scale industrial applications.

REFERENCES

[1] D. Bytschkow, M. Zellner and M. Duchon, "Combining SCADA, CIM, GridLab-D and AKKA for smart grid **co**-simulation," *2015 IEEE Power & Energy Society Innovative Smart Grid Technologies Conference (ISGT)*, Washington, DC, USA, 2015, pp. 1-5, doi: 10.1109/ISGT.2015.7131872.

[2] H. lv, X. Ge, H. Zhu, C. Wang, Z. Yuan and Y. Zhu, "Design and Implementation of Reactive Distributed Internet of Things Platform based on Actor Model," *2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)*, Chengdu, China, 2019, pp. 1993-1996, doi: 10.1109/ITNEC.2019.8729169.

[3] Debopama Sen Sarma, Sebastian Peter , Christian Rehtanz, "A Distributed Framework for Agent-based Optimal Energy Management of Distribution Systems" *2023 IEEE PES Innovative Smart Grid Technologies Europe (ISGT EUROPE) | 979-8-3503-9678-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/ISGTEUROPE56780.*2023.10407568

[4] A. Huber, D. T. Schroeder, K. Pogorelov, C. Griwodz and J. Langguth, "A Streaming System for Large-scale Temporal Graph Mining of Reddit Data," *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, Lyon, France, 2022, pp. 1153-1162, doi: 10.1109/IPDPSW55747.2022.00189.

[5] F Ghassemi, M Sirjani, E Khamespanah, M Mirani, H Hojjat, Transparent Actor Model 2023 IEEE/ACM 11th International Conference on Formal Methods in Software Engineering (FormaliSE) | 979-8-3503-1263-8/23/$31.00 ©2 023 IEEE | DOI: 10.1109/FormaliSE58978.2023.00018

[6] J. De Bleser, D. Di Nucci and C. De Roover, "A Delta-Debugging Approach to Assessing the Resilience of Actor Programs through Run-time Test Perturbations," *2020 IEEE/ACM 15th International Conference on Automation of Software Test (AST)*, Seoul, Korea, Republic of, 2020, pp. 21-30.

[7] S. Das, S. Goswami, and S. C N, "Optimizing Apache Kafka Deployments - Configuration customization is all you need," *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, Delhi, India, 2023, pp. 1-9, doi: 10.1109/ICCCNT56998.2023.10307242.

[8] G. Fu, Y. Zhang, and G. Yu, "A Fair Comparison of Message Queuing Systems," in IEEE Access, vol. 9, pp. 421-432, 2021, doi: 10.1109/ACCESS.2020.3046503.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:42:39 UTC from IEEE Xplore. Restrictions apply.